Project Proposal: DecompGTI & MCP Integration

Augmenting Small LLMs with Decomposed Graph Tool Instruction

Arpna Gupta
IIT Gandhinagar
24110051@iitgn.ac.in

Buha Deep
IIT Gandhinagar
24110082@iitgn.ac.in

Rathod Yuvraj
IIT Gandhinagar
yuvraj.rathod@iitgn.ac.in

Solanki Viraj
IIT Gandhinagar
viraj.solanki@iitgn.ac.in

Abstract

Small Large Language Models (LLMs) under 8B parameters severely underperform on graph-structured data due to their inability to reliably track complex topologies. We propose DecompGTI, a framework that augments small LLMs with Decomposed Graph Tool Instruction by integrating a Model Context Protocol (MCP) server. By offloading deterministic graph computations to an external Python environment through a strict, self-correcting three-step pipeline, DecompGTI enables highly accurate graph reasoning and algorithmic execution on consumer-grade hardware.

1 Motivation and Problem Statement

Current large language models (LLMs) severely underperform on graph-structured data due to inherent limitations in tracking complex topologies within their attention mechanisms, often hallucinating edges or failing multi-step logical inference. Existing Tool-Instruction methods attempt to solve this by teaching models to call external APIs, but they fail on small language models (under 8B parameters) because they skip the semantic understanding of the graph itself.

Our primary goal is to implement and validate the GraphTool-Instruction framework augmented by a Model Context Protocol (MCP) server. Instead of relying on the LLM's internal mathematical capabilities, we will design an agentic routing system that offloads strict algorithmic computation to an external Python environment.

2 System Architecture: The 3-Step Pipeline

To bridge the gap between natural language prompts and deterministic graph algorithms, we will fine-tune a small LLM to execute a strictly decomposed reasoning pipeline:

(1) **Graph Extraction:** The LLM parses natural language (e.g., "Node A connects to B and C") into a deterministic JSON format representing an adjacency list. This format has been empirically proven to yield the highest structural comprehension for LLMs.
(2) **Tool Name Identification:** The model classifies the user's implicit intent and routes it to the correct algorithmic tool (e.g., mapping a bottleneck query to a Maximum Flow algorithm).
(3) **Tool Parameter Extraction:** The model extracts and formats the precise arguments (e.g., source node, target node, edge weights) required by the target tool's strict JSON schema.

3 MCP Server Specification & Tool Registry

The MCP server will act as the deterministic execution engine, wrapping optimized graph algorithms from the `NetworkX` library. To cover varying levels of complexity, the tool registry will include implementations standard in advanced competitive programming:

- **Traversal & Paths:** Breadth-First Search (BFS), Depth-First Search (DFS), and Dijkstra's Algorithm for shortest path routing.
- **Network Flow:** Edmonds-Karp or Dinic's Algorithm for Maximum Flow problems.
- **Structural Analysis:** Bipartite Maximum Matching, Topological Sorting, and Cycle Detection.
- **Optimization:** Kruskal's or Prim's Algorithm for Minimum Spanning Trees (MST).

Each tool will enforce strict input validation. If the LLM generates a type mismatch (e.g., outputting a string when an integer node ID is required), the MCP server will return an execution error, forcing the LLM to self-correct.

4 Dataset and Methodology

Dataset Selection: We will utilize the *GraphInstruct* dataset, which encompasses 21 classical graph reasoning tasks at varying levels of difficulty. We will use the "adjacency tables in natural language" data split to train the extraction phase.

Usage Strategy: The raw training data will be reformatted into our decomposed instruction format. The target outputs will not be the final answers, but rather the step-by-step intermediate tokens required for tool selection and parameter extraction.

5 Evaluation and Benchmarking

We will rigorously benchmark the DecompGTI approach against the fine-tuned *GraphSolver* model baseline, which operates without external tools.

- **Target Models:** Open-weights models under 8B parameters (e.g., LLaMA-3.1-8B-Instruct or Qwen-2.5-7B-Instruct).
- **Primary Metrics:**
  - **Tool Identification Accuracy:** Exact match rate for routing to the correct algorithm.
  - **Parameter Extraction Score:** Precision and recall measuring the schema compliance of the extracted arguments.
  - **Task Success Rate:** Exact-match accuracy of the final answer post-MCP execution, evaluated across varying

graph sizes (mini: 5-7 nodes, small: 8-15, medium: 16-25, large: 26-35) to test context window resilience and scalability.

6 Computational Infrastructure

To ensure the project remains practical for local execution and iteration on consumer-grade hardware (e.g., a modern laptop with a dedicated mid-tier GPU), we will utilize Parameter-Efficient Fine-Tuning (PEFT) techniques, specifically QLoRA. This allows us to train the <8B parameter models efficiently without requiring enterprise-grade cluster compute.

7 Use Cases

The proposed system can be applied to various real-world problems that involve graph-structured data and require reasoning over relationships.

- **Navigation and Route Planning:** The system can determine the shortest or optimal path between locations in a road network, similar to map-based applications.
- **Social Network Analysis:** It can analyze relationships between individuals, such as finding mutual connections or identifying highly connected nodes.
- **Knowledge Graph Question Answering:** The system can answer queries about relationships between entities by constructing and reasoning over knowledge graphs.
- **Network Connectivity Analysis:** It can evaluate whether a network is fully connected and identify critical nodes or edges.
- **Supply Chain Optimization:** The system can assist in identifying efficient routes and detecting bottlenecks in logistics networks.

8 Timeline

- **Week 1:** System setup, dataset preparation, graph schema design, implementation of graph extraction and basic MCP server, integration of core graph tools.
- **Week 2:** Development of tool selection and parameter extraction modules, addition of graph operations, testing and debugging.
- **Week 3:** Evaluation, comparison with baseline methods, error analysis, and preparation of final report and presentation.

9 Expected Outcome

The project is expected to produce a working system that can take simple graph-related queries in natural language and return correct answers using graph-processing tools. The system will demonstrate the ability to break down complex problems into smaller steps such as graph extraction, tool selection, and execution.

The final outcome will include a prototype, test examples, and a demonstration showing how the proposed approach improves the accuracy of graph-based reasoning compared to a direct method.